Linear Grammars

Grammars with
at most one variable at the right side
of a production

Examples:

$$S \to aSb$$
$$S \to \lambda$$

$$S \to Ab$$
$$A \to aAb$$
$$A \to \lambda$$

A Non-Linear Grammar

Grammar G :

$$S \to SS$$

$$S \to \lambda$$

$$S \to aSb$$

$$S \to bSa$$

$$L(G) = \{w : \ n_a(w) = n_b(w)\}$$

Number of a in string w

Another Linear Grammar

Grammar G :

$$S \to A$$

$$A \to aB \mid \lambda$$

$$B \to Ab$$

$$L(G) = \{a^n b^n : n \geq 0\}$$

Right-Linear Grammars

All productions have form: $A \rightarrow xB$

or

$A \rightarrow x$

string of terminals

Example: $S \rightarrow abS$

$S \rightarrow a$

Left-Linear Grammars

All productions have form: $A \to Bx$

or

$A \to x$

string of terminals

Example: $S \to Aab$

$A \to Aab \mid B$

$B \to a$

Regular Grammars

Regular Grammars

A regular grammar is any
right-linear or left-linear grammar

Examples:

G_1

$$S \to abS$$

$$S \to a$$

G_2

$$S \to Aab$$

$$A \to Aab \mid B$$

$$B \to a$$

Observation

Regular grammars generate regular languages

Examples:

G_1

$S \to abS$

$S \to a$

$L(G_1) = (ab)^* a$

G_2

$S \to Aab$

$A \to Aab \mid B$

$B \to a$

$L(G_2) = aab(ab)^*$

Regular Grammars Generate Regular Languages

Theorem

$$\left\{\begin{array}{l}\text{Languages}\\ \text{Generated by}\\ \text{Regular Grammars}\end{array}\right\} = \left\{\begin{array}{l}\text{Regular}\\ \text{Languages}\end{array}\right\}$$

Theorem - Part 1

$$\left\{ \begin{array}{l} \text{Languages} \\ \text{Generated by} \\ \text{Regular Grammars} \end{array} \right\} \subseteq \left\{ \begin{array}{l} \text{Regular} \\ \text{Languages} \end{array} \right\}$$

Any regular grammar generates a regular language

Theorem - Part 2

$$\left\{\begin{array}{l}\text{Languages}\\\text{Generated by}\\\text{Regular Grammars}\end{array}\right\} \supseteq \left\{\begin{array}{l}\text{Regular}\\\text{Languages}\end{array}\right\}$$

Any regular language is generated by a regular grammar

Proof - Part 1

$$\left\{\begin{array}{l}\text{Languages}\\ \text{Generated by}\\ \text{Regular Grammars}\end{array}\right\} \subseteq \left\{\begin{array}{l}\text{Regular}\\ \text{Languages}\end{array}\right\}$$

The language $L(G)$ generated by any regular grammar G is regular

The case of Right-Linear Grammars

Let G be a right-linear grammar

We will prove: $L(G)$ is regular

Proof idea: We will construct NFA M with $L(M) = L(G)$

Grammar G is right-linear

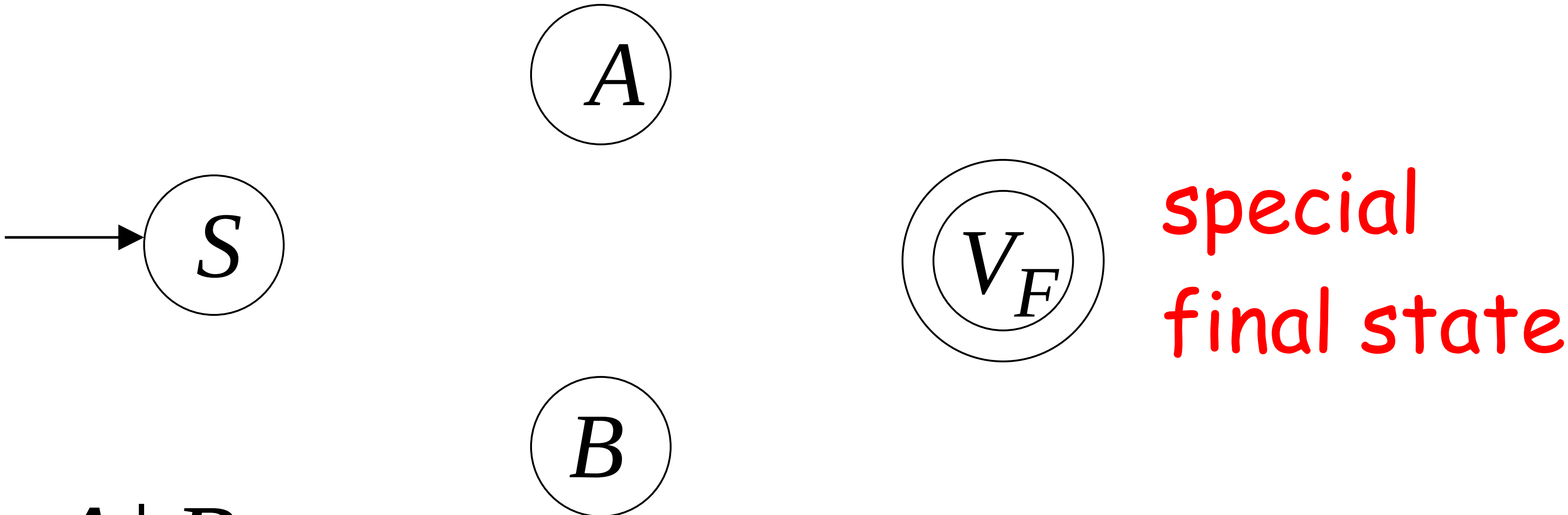
Example:

$$S \to aA \mid B$$

$$A \to aa\,B$$

$$B \to b\,B \mid a$$

Construct NFA M such that every state is a grammar variable:

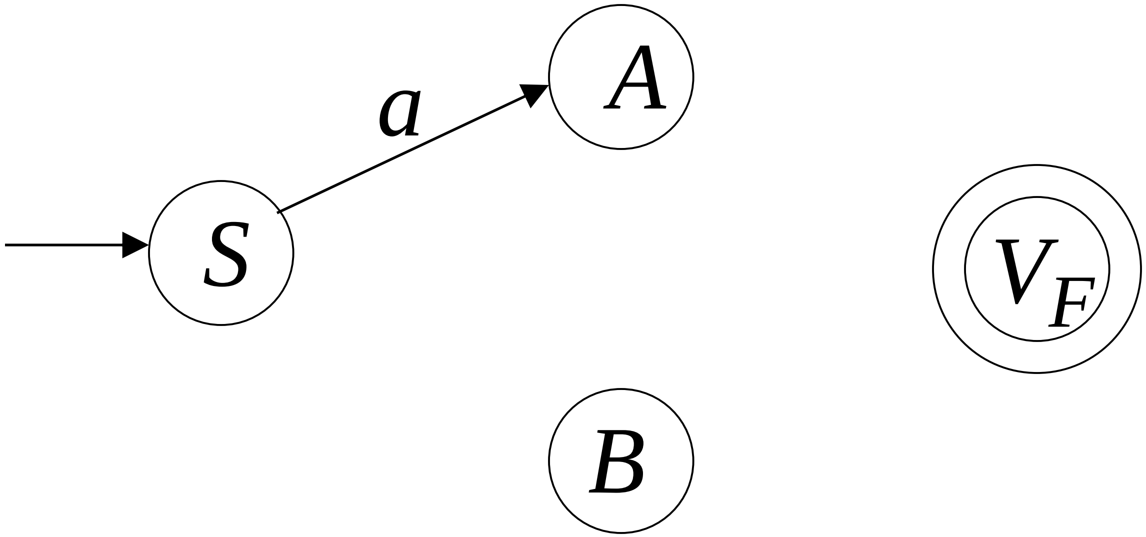


$$S \to aA \mid B$$

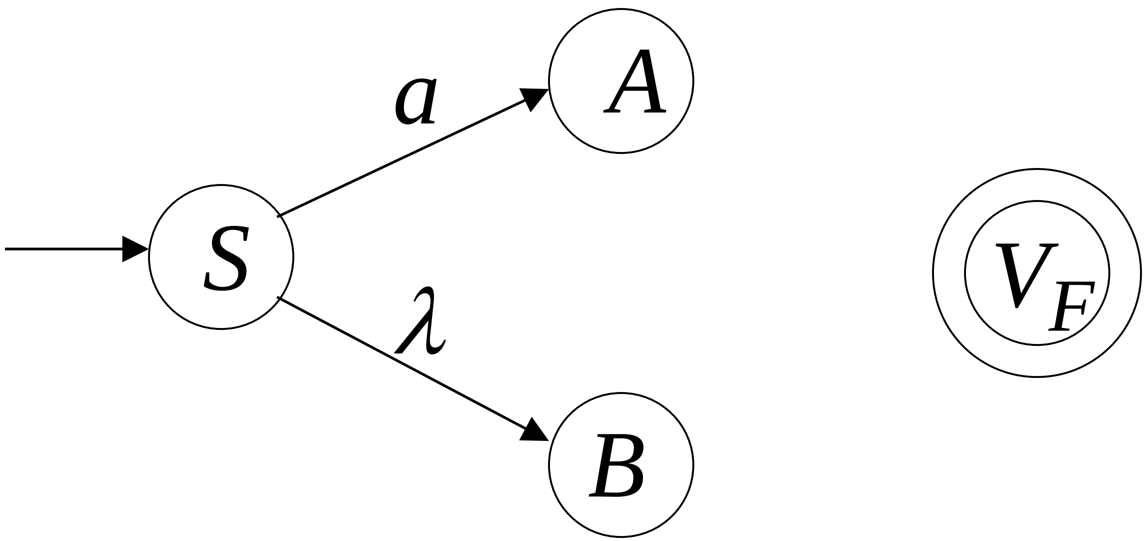
$$A \to aa\,B$$

$$B \to b\,B \mid a$$

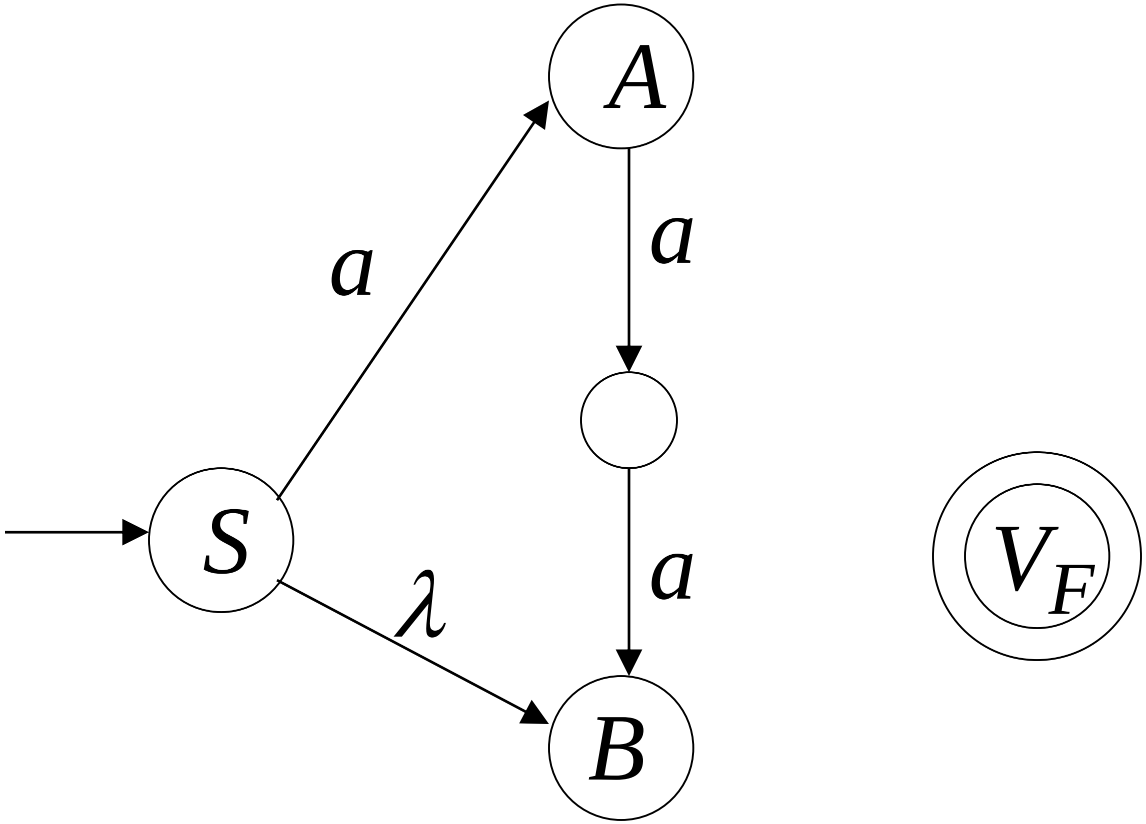
Add edges for each production:



$$S \to aA$$

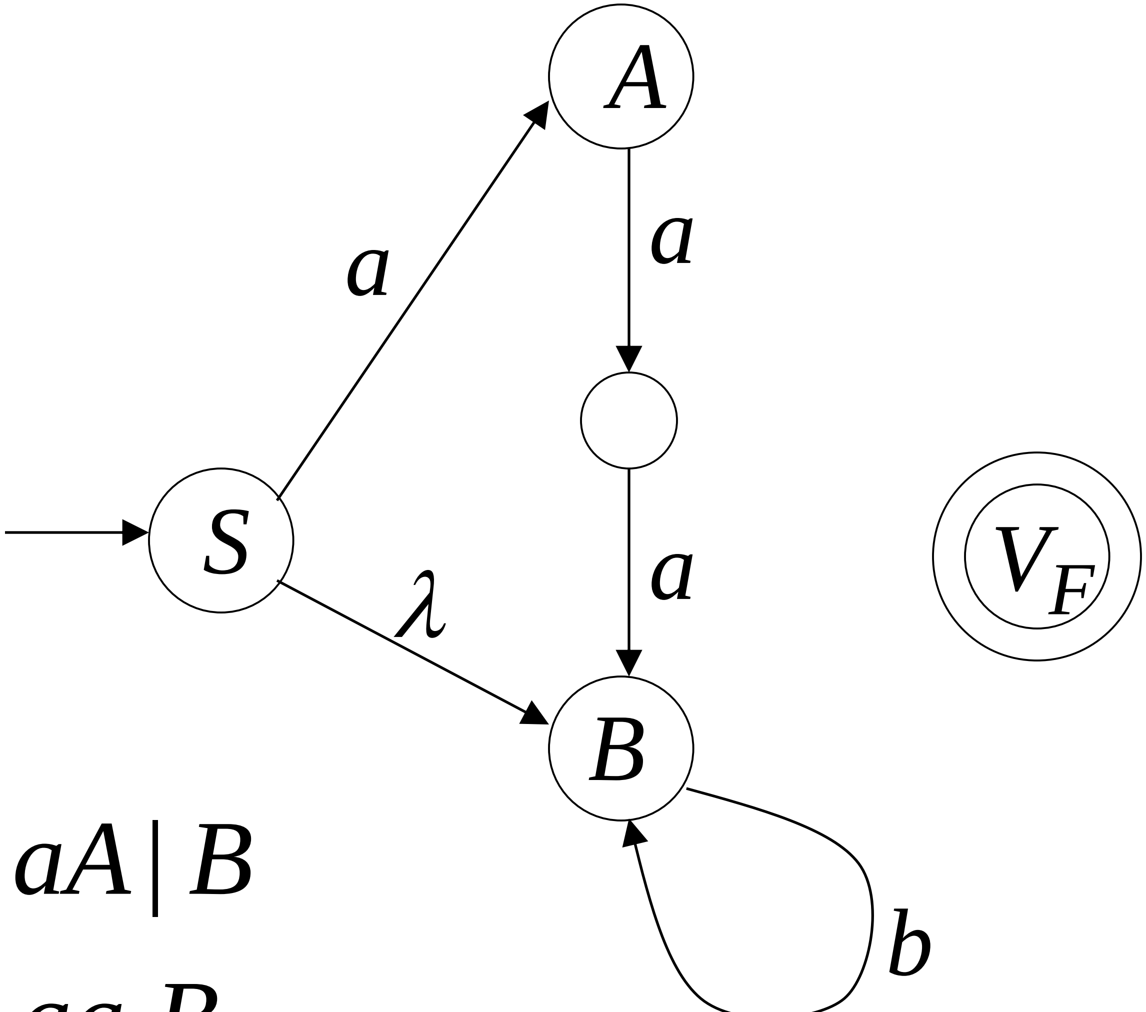


$$S \to aA \mid B$$



$$S \to aA \mid B$$

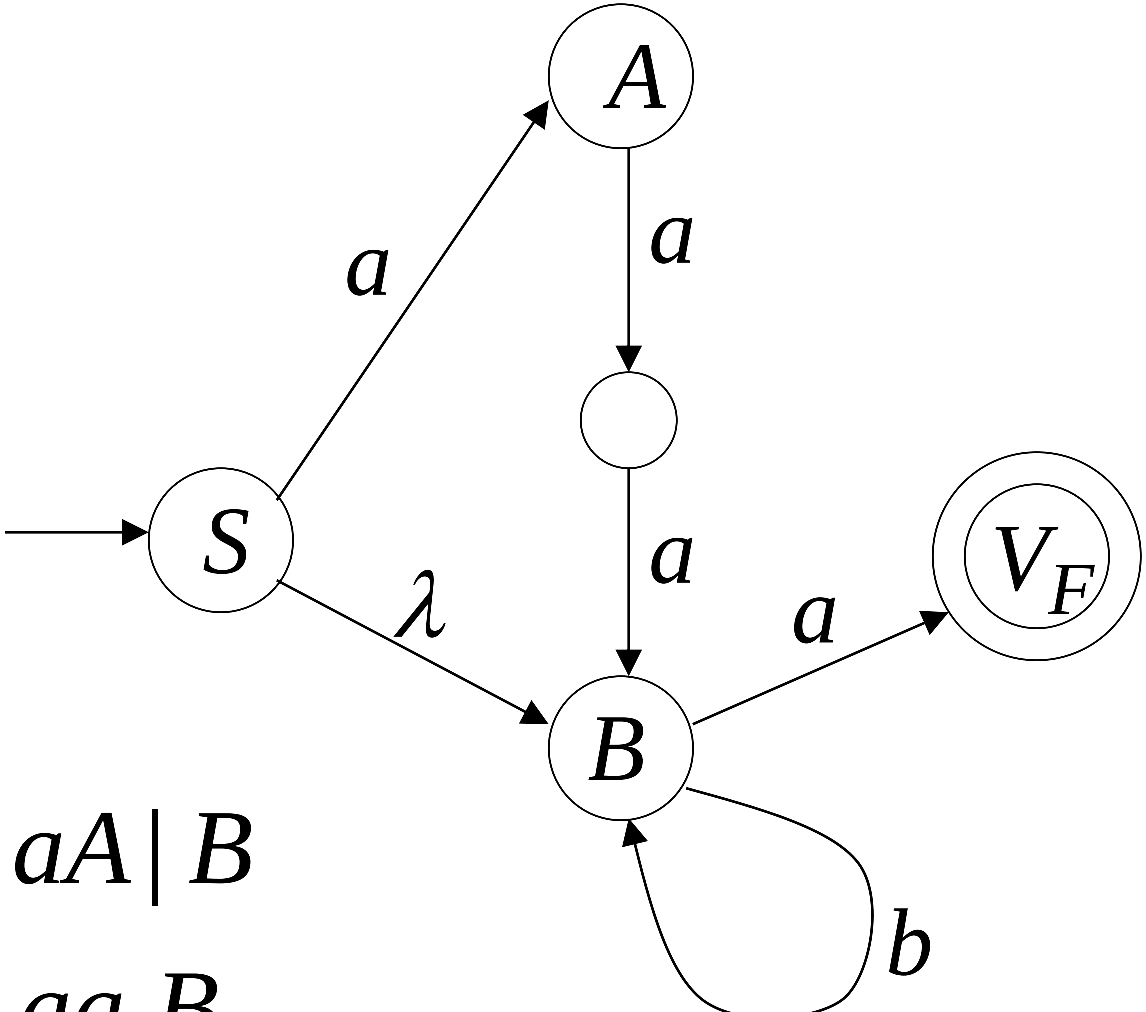
$$A \to aa\, B$$



$$S \to aA \mid B$$

$$A \to aa\,B$$

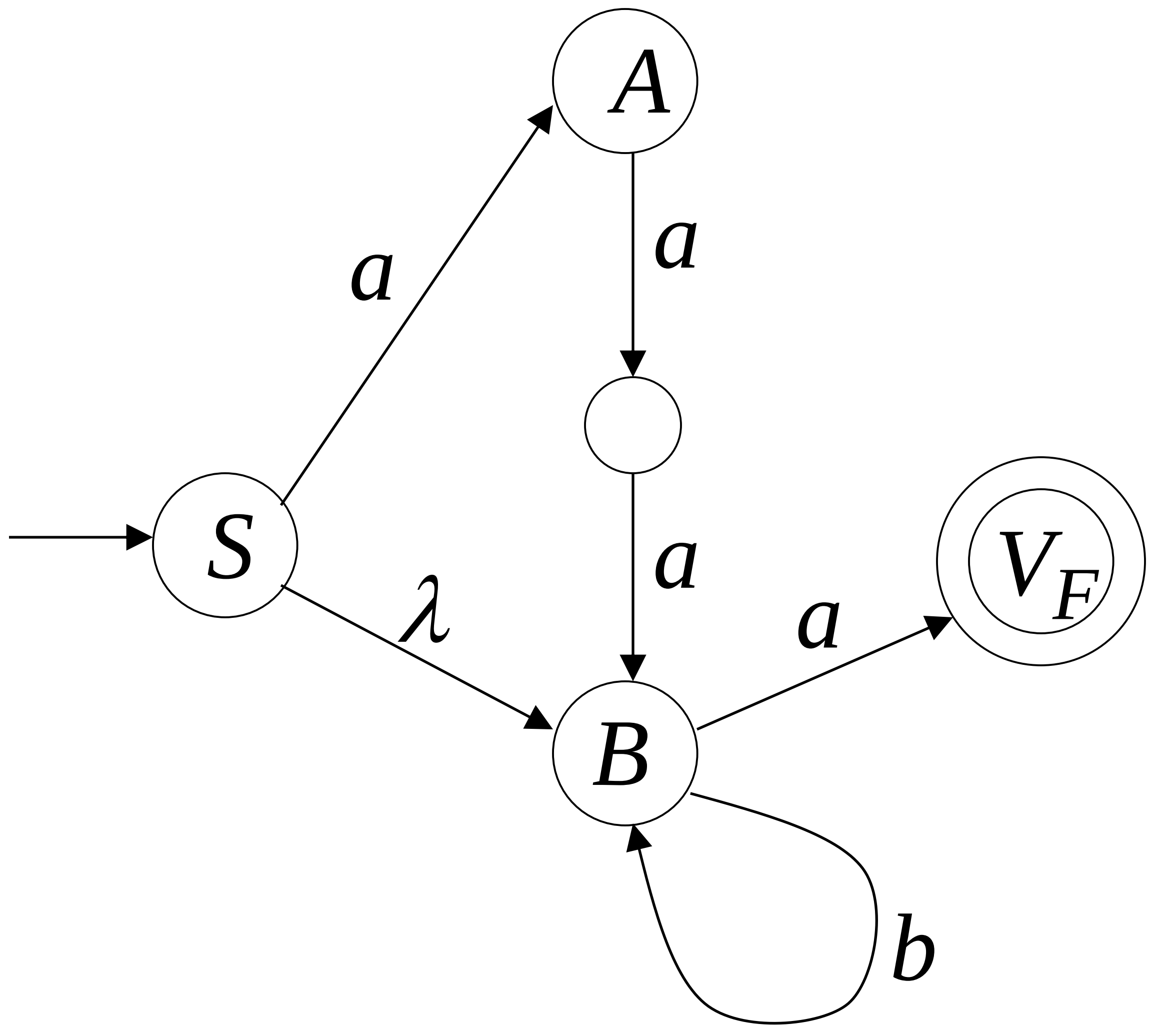
$$B \to bB$$



$$S \to aA \mid B$$

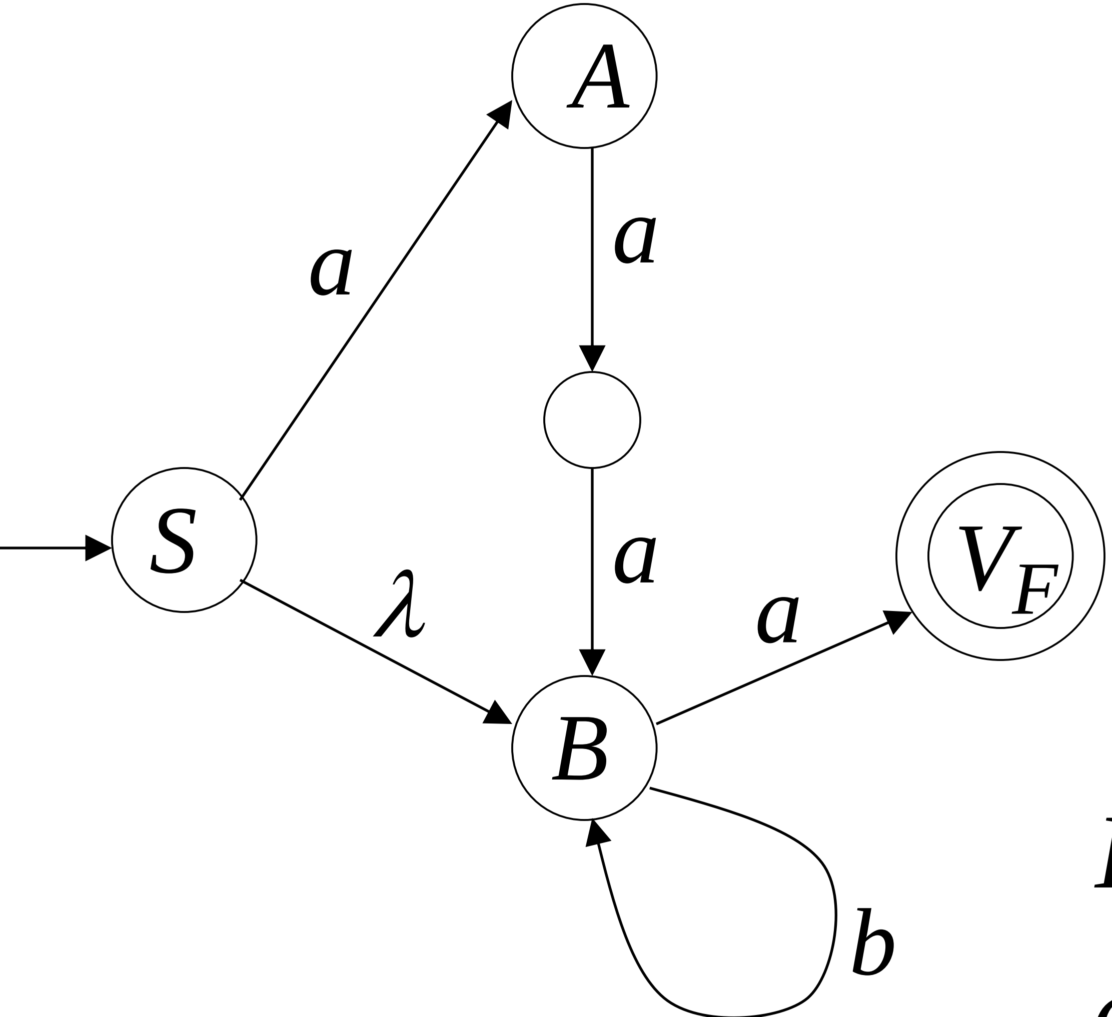
$$A \to aa\ B$$

$$B \to bB \mid a$$



$$S \Rightarrow aA \Rightarrow aaaB \Rightarrow aaabB \Rightarrow aaaba$$

NFA M



Grammar G

$$S \to aA \mid B$$

$$A \to aa\,B$$

$$B \to bB \mid a$$

$$L(M) = L(G) =$$

$$aaab^*a + b^*a$$

In General

A right-linear grammar G

has variables: $V_0, V_1, V_2, \ldots$

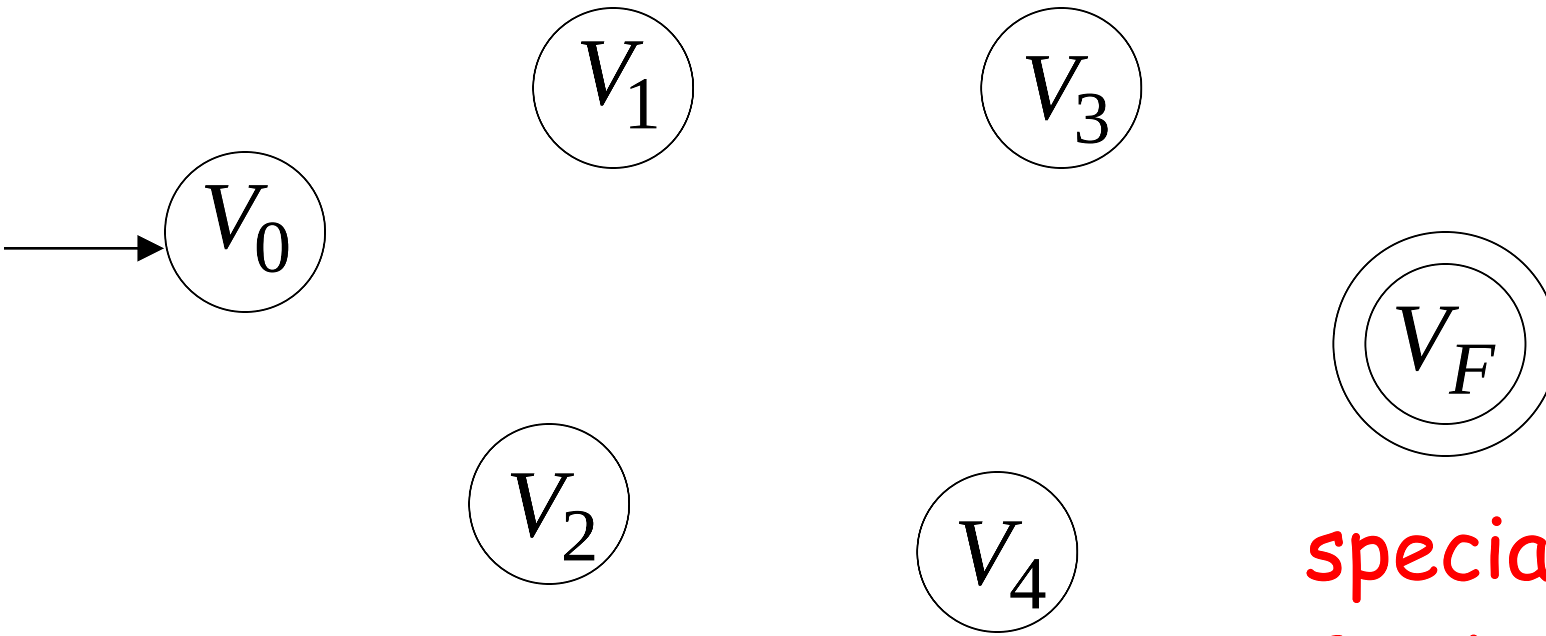
and productions: $V_i \rightarrow a_1 a_2 \cdots a_m V_j$

or

$$V_i \rightarrow a_1 a_2 \cdots a_m$$

We construct the NFA M such that:

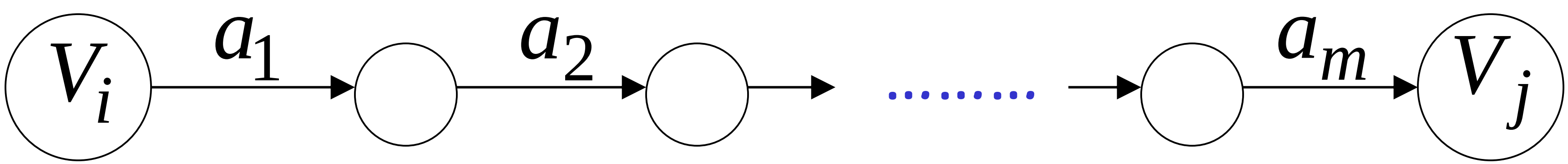
each variable V_i corresponds to a node:



special final state

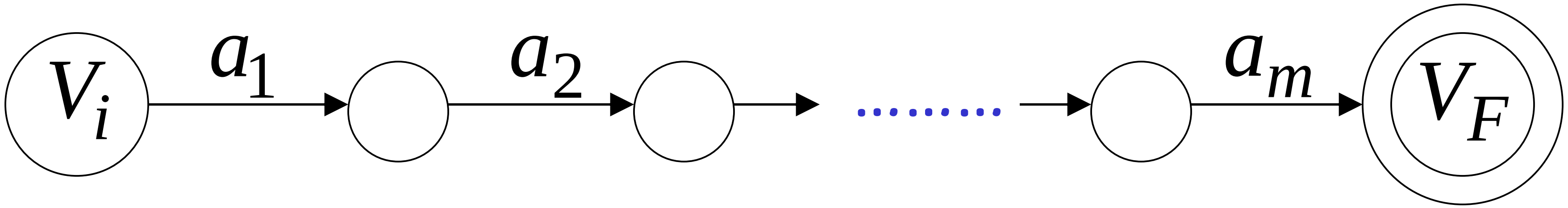
For each production: $V_i \rightarrow a_1 a_2 \cdots a_m V_j$

we add transitions and intermediate nodes

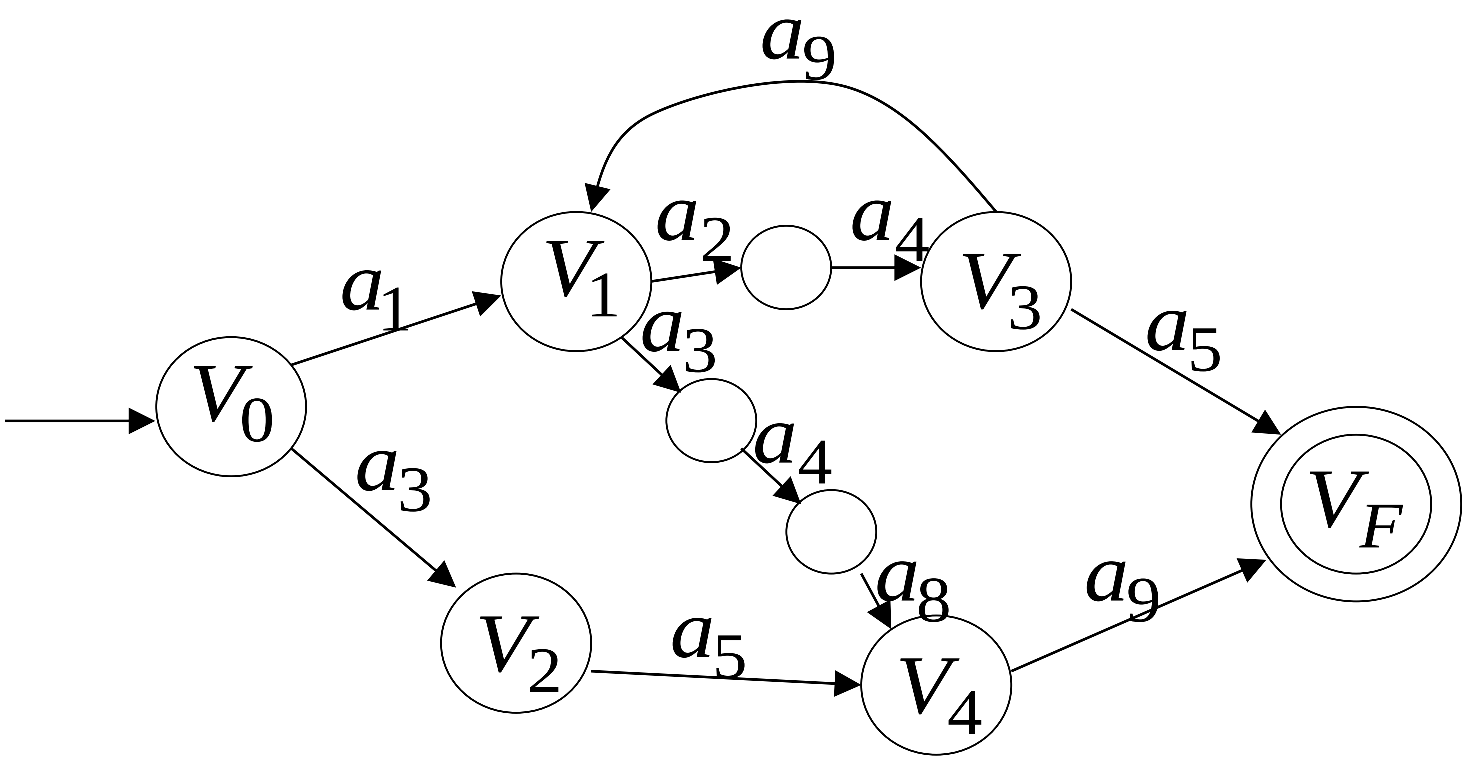


For each production: $V_i \to a_1 a_2 \cdots a_m$

we add transitions and intermediate nodes



Resulting NFA M looks like this:



It holds that: $L(G) = L(M)$

The case of Left-Linear Grammars

Let G be a left-linear grammar

We will prove: $L(G)$ is regular

Proof idea:

We will construct a right-linear grammar G' with $L(G) = L(G')^R$

Since G is left-linear grammar the productions look like:

$$A \to Ba_1 a_2 \cdots a_k$$

$$A \to a_1 a_2 \cdots a_k$$

Construct right-linear grammar G'

Left linear G

$$A \to Ba_1 a_2 \cdots a_k$$

$$A \to Bv$$

Right linear G'

$$A \to a_k \cdots a_2 a_1 B$$

$$A \to v^R B$$

Construct right-linear grammar G'

Left linear G

$$A \to a_1 a_2 \cdots a_k$$

$$A \to v$$

Right linear G'

$$A \to a_k \cdots a_2 a_1$$

$$A \to v^R$$

It is easy to see that: $L(G) = L(G')^R$

Since G' is right-linear, we have:

$L(G') \Rightarrow L(G')^R \Rightarrow L(G)$

Regular Language | Regular Language | Regular Language

Proof - Part 2

$$\left\{\begin{array}{l}\text{Languages} \\ \text{Generated by} \\ \text{Regular Grammars}\end{array}\right\} \supseteq \left\{\begin{array}{l}\text{Regular} \\ \text{Languages}\end{array}\right\}$$

Any regular language L is generated by some regular grammar G

Any regular language L is generated by some regular grammar G

Proof idea:

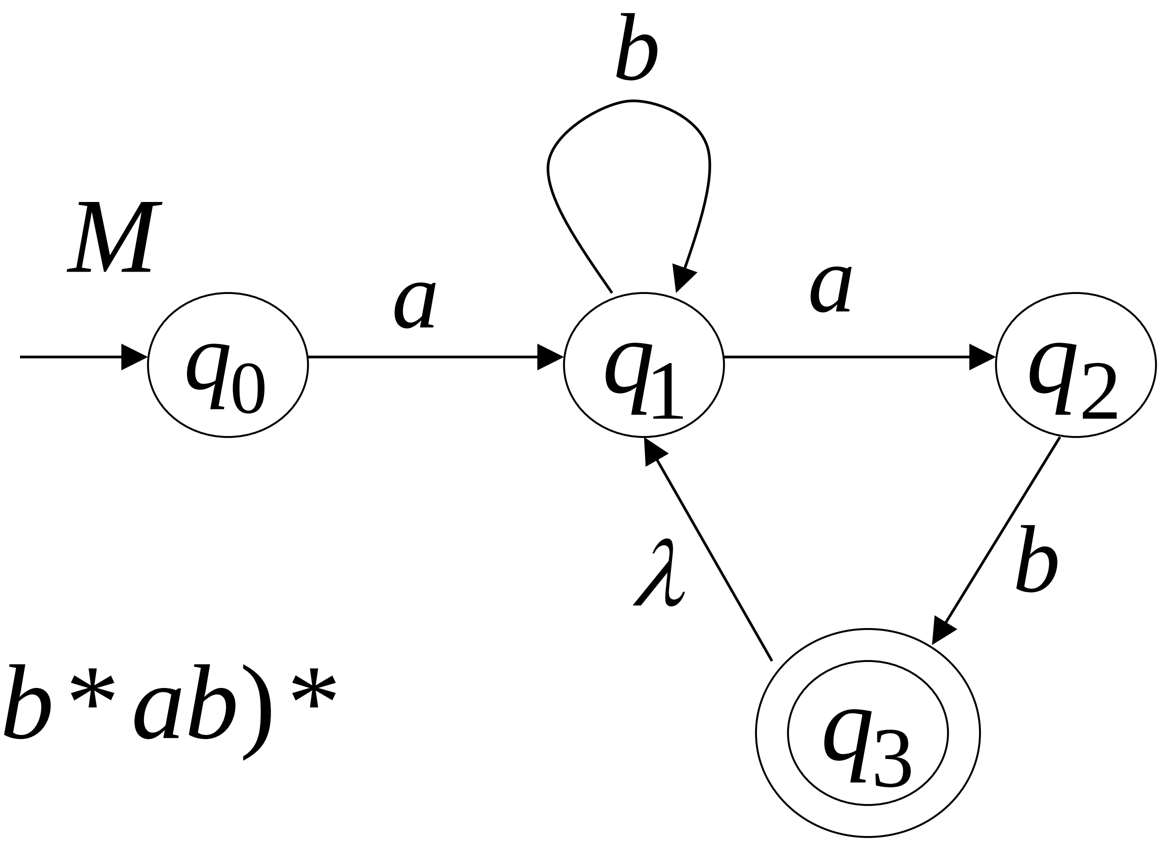
Let M be the NFA with $L = L(M)$.

Construct from M a regular grammar G such that $L(M) = L(G)$

Since L is regular

there is an NFA M such that $L = L(M)$

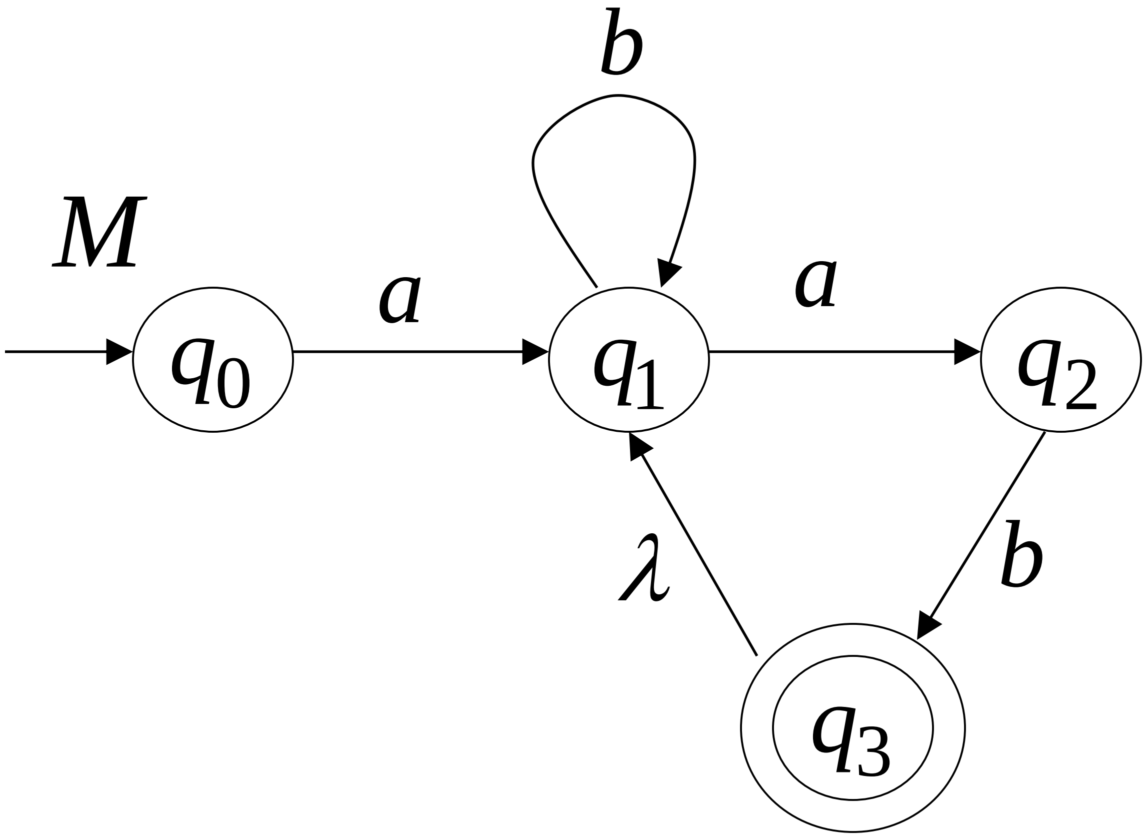
Example:



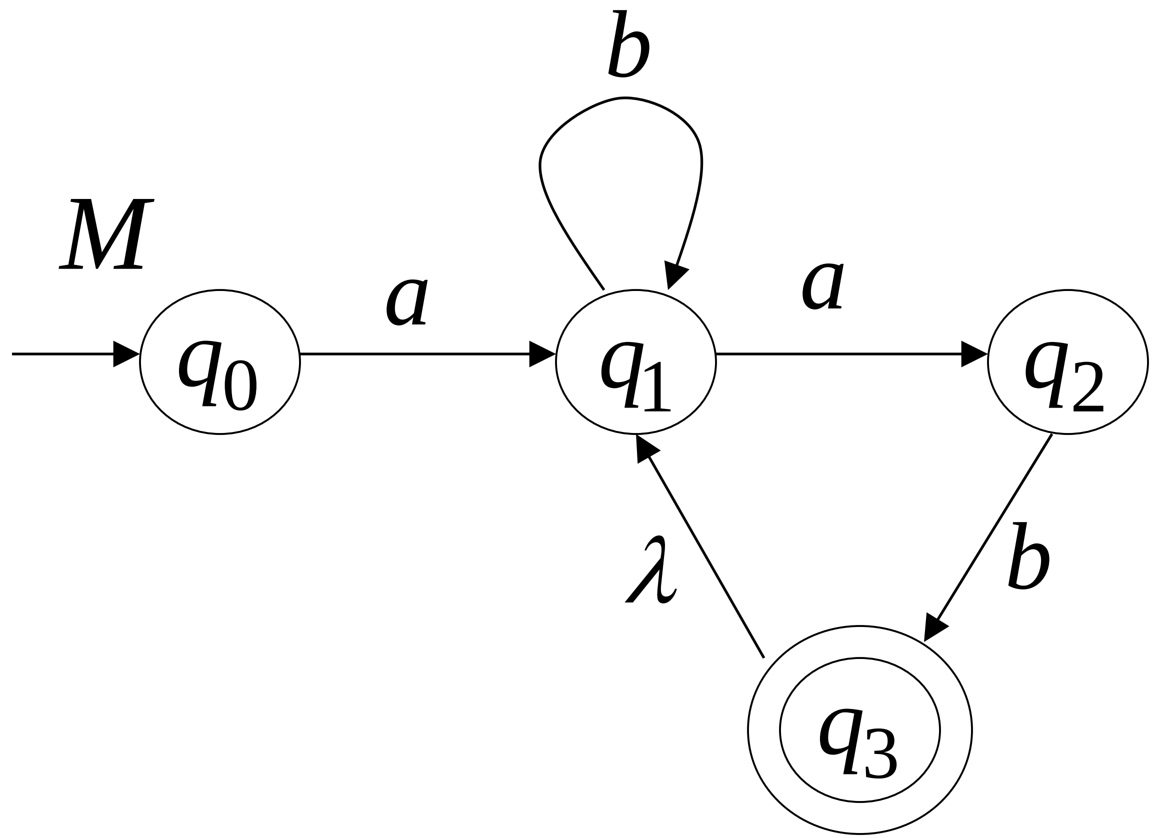
$$L = ab^*ab(b^*ab)^*$$

$$L = L(M)$$

Convert M to a right-linear grammar



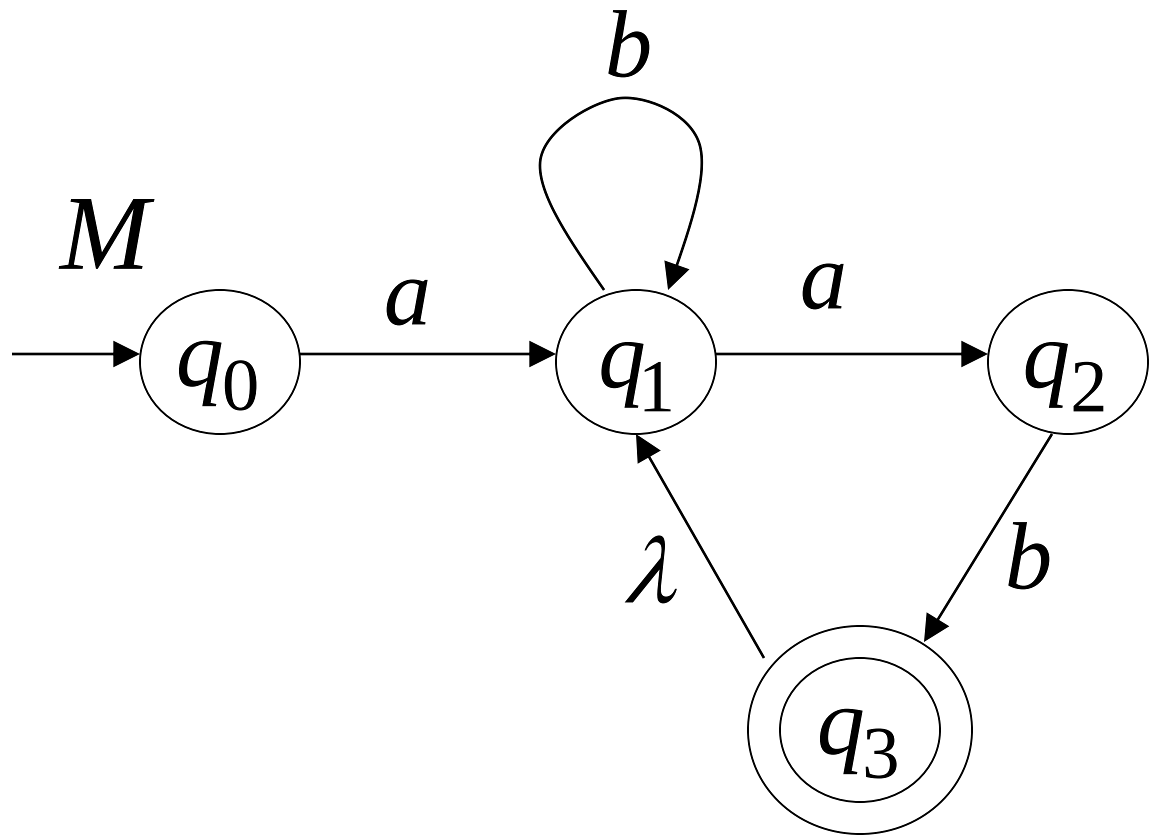
$$q_0 \to aq_1$$



$$q_0 \to aq_1$$

$$q_1 \to bq_1$$

$$q_1 \to aq_2$$



$$q_0 \to aq_1$$

$$q_1 \to bq_1$$

$$q_1 \to aq_2$$

$$q_2 \to bq_3$$

$$L(G) = L(M) = L$$

G

$$q_0 \to aq_1$$

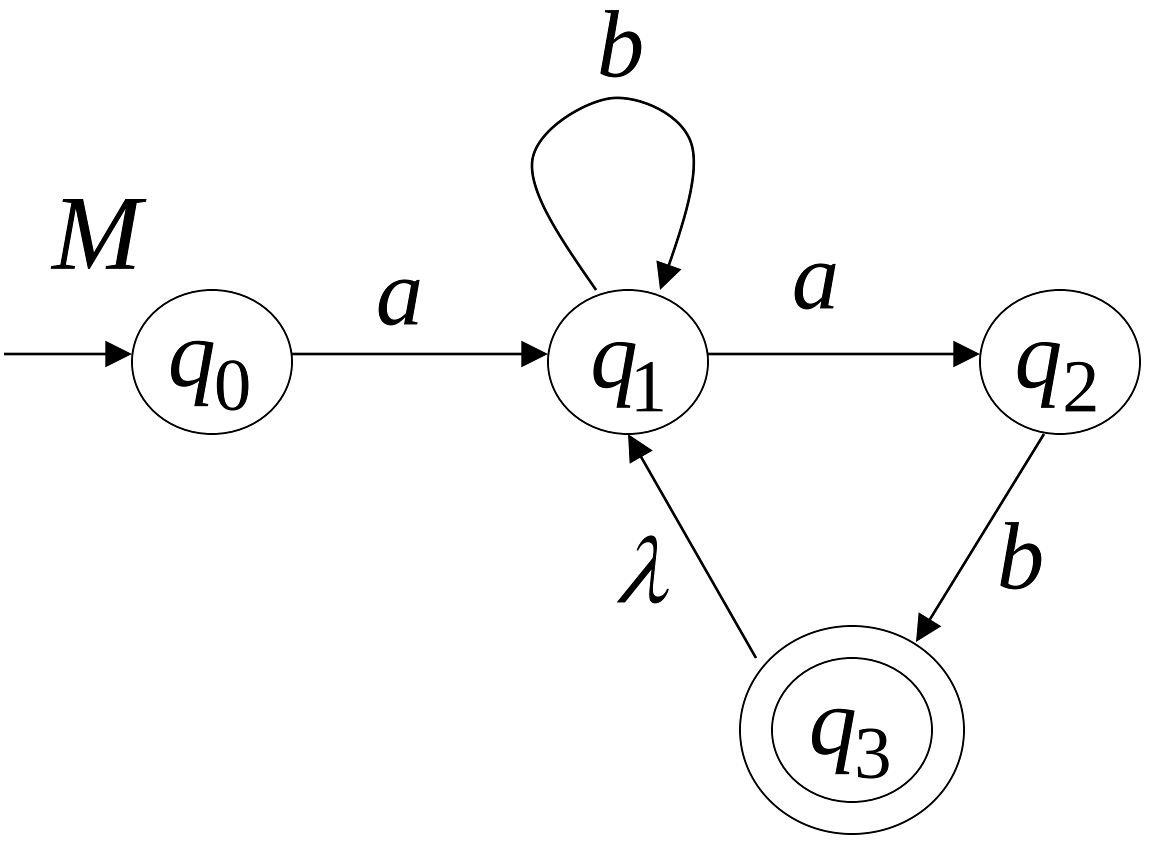
$$q_1 \to bq_1$$

$$q_1 \to aq_2$$

$$q_2 \to bq_3$$

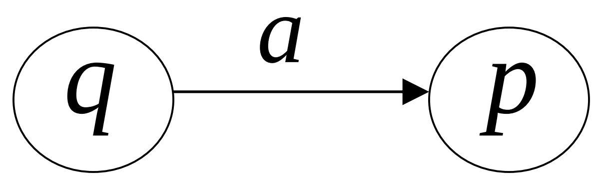
$$q_3 \to q_1$$

$$q_3 \to \lambda$$



In General

For any transition:

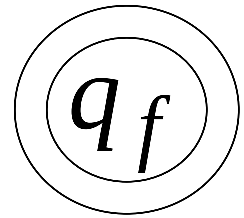


Add production:

$$q \to ap$$

variable terminal variable

For any final state:



Add production: $q_f \to \lambda$

Since G is right-linear grammar

G is also a regular grammar

with $L(G) = L(M) = L$

Pumping lemma

- This theorem can be used to prove that certain sets are not regular.
- We now give the steps needed for proving that a given set is not regular.

Step 1: Assume L is regular. Let 'n' be the number of states in the corresponding F.A.

Step 2: choose a string w such that |w|>=n.Use pumping lemma to write,w =xyz,with |xy| <=n and |y|>0

Step 3: Find suitable integer 'i' such that xy^iz should not belongs to L.

Pumping Lemma

- Pumping Lemma is used to prove that a Language is NOT REGULAR
- It cannot be used to prove that a Language is Regular

If A is a Regular Language, then A has a Pumping Length 'P' such that any string 'S' where $|S| \geq P$ may be divided into 3 parts $\underline{S = x\,y\,z}$ such that the following conditions must be true:

(1) $x\,y^i z \in A$ for every $i \geq 0$

(2) $|y| > 0$

(3) $|xy| \leq P$

To prove that a language is not Regular using PUMPING LEMMA, follow the below steps:

(We prove using Contradiction)

- -> Assume that A is Regular
- -> It has to have a Pumping Length (say P)
- -> All strings longer than P can be pumped $|S| \geq P$
- -> Now find a string 'S' in A such that $|S| \geq P$
- -> Divide S into x y z
- -> Show that $x\,y^i z \notin A$ for some i
- -> Then consider all ways that S can be divided into x y z
- -> Show that none of these can satisfy all the 3 pumping conditions at the same time
- -> S cannot be Pumped == CONTRADICTION

Example 1

Shoe that the set,L={a^n |n>=0} is regular or not.

W=aa

Case 1:

X=null

Y=aa

Z=null

For all value of i, still belongs to language.

Example 2

Shoe that the set,L={a^2n |n>=0} is regular or not.

W=aa

Case 1:

X=null

Y=aa

Z=null

For all value of i, still belongs to language.

Example 3

Shoe that the set,L={a^nb^n |n>0} is regular or not.

W=ab

Case 1:

X=a

Y=b

Z=null

For all value of i,

When i=1 result will be still belongs to language.

When i=2 result will be abb ,string not belongs to language ,hence proved language is not regular

Shoe that the set,L={a^nb^nc^n |n>0} is regular or not.

W=abc

Case 1:

X=a

Y=b

Z=c

For all value of i,

When i=1 result will be still belongs to language.

When i=2 result will be abbc ,string not belongs to language ,hence proved language is not

Pumping lemma

Apply Pumping Lemma on languages **L1** ,**L2** and **L3** to prove that these are not regular languages.

(i) $L1=\{0^n 1^m : n \le m+3\}$ **(ii)** $L2=\{01(1100)^n 110(10)^n : n \ge 0\}$

(iii) $L3 = \{a^i b^j c^k : i >= j >= k >= 1\}$

Pumping lemma

Is the language

L = {w | w is a binary string with at least as many 1's as 0's}

regular? Prove your answer using pumping lemma

Bilal, a student of CS301, has submitted following answer:

I will prove that L is not regular, using the pumping lemma.

i. Let p is a pumping length

ii. Let w = 01 p , so that w ∈ L and |w| ≥ p.

iii. Let w = xyz, with x = ε, y = 0, and z = 1 p , so that |xy| ≤ p and |y| > 0.

iv. Then xy p+1 z is not in L.

Identify the single incorrect line in Bilal's proof and explain what he did wrong.

(NOTE: do not list small mistakes, typos, etc.; explain the major logical error which invalidates Bilal's

proof.)

Solution

In line iii., Bilal did wrong split of w. As 0 is finite in w, therefore cannot be pumped and hence invalidate

the proof. In another words, Bilal is not permitted to pick a specific split of w.

b. “Let L be any finite, nonempty language of binary strings. Is L regular?”

Jameel has submitted the following answer:

I will prove that L is not regular, using the pumping lemma.

i. Since L is finite, it has a longest string. Let p be the length of the

longest string in L.

ii. Let $w \in L$, such that $|w| \geq p$.

iii. Let w = xyz, so that |xy| ≤ p and |y| > 0.

iv. Then xy 2 z is not in L, since it has length longer than p, but all the

strings in L have length at most p.

Identify the single incorrect line in Jameel’s proof, and explain what he did wrong.

(NOTE: Do not just say, “L is actually regular”; explain the major logical error which invalidates Jameel’s

proof.)